

Parallel Image Convolution

Analysis Report

High Performance Computing (HPC)
7th Semester — BSc. Engineering

Implementations: Serial, OpenMP, POSIX Pthreads, MPI, CUDA, MPI+OpenMP
Filters Tested: Gaussian Blur, Edge Detection, Sharpen
Repository: Parallel-Image-Convolution
Date: May 2026

Team Members

Imalsha Ranasinghe

Contents

1	Introduction	2
2	Parallel Programming Concepts Applied	2
2.1	Overview of the Parallelisation Strategy	2
2.2	Serial Baseline	2
2.3	OpenMP — Shared-Memory Thread Parallelism	3
2.4	POSIX Pthreads — Explicit Thread Management	4
2.5	MPI — Distributed-Memory Parallelism	4
2.6	CUDA — GPU Massive Parallelism	5
2.7	Hybrid MPI + OpenMP — Two-Level Parallelism	6
2.8	Summary of Parallelisation Concepts	8
3	Accuracy Analysis	8
3.1	Metric: Root Mean Square Error (RMSE)	8
3.2	Expected Sources of Error by Implementation	8
3.3	RMSE Results	9
4	Performance Analysis	10
4.1	Experimental Setup	10
4.2	Gaussian Blur — Execution Time vs. Thread/Process Count	10
4.3	Speedup and Amdahl’s Law	12
4.4	Hybrid MPI+OpenMP Configurations	12
4.5	CUDA Block-Size Sensitivity	13
4.6	Edge Detection and Sharpen (3×3)	14
4.7	Memory Bandwidth Bottleneck	14
4.8	Parallel Efficiency	15
5	Conclusion	15

1 Introduction

Image convolution is a fundamental operation in computer vision and image processing. Given an input image I and a convolution kernel K of size $k \times k$, the output pixel at position (x, y) and colour channel c is:

$$O(x, y, c) = \sum_{ky=-\lfloor k/2 \rfloor}^{\lfloor k/2 \rfloor} \sum_{kx=-\lfloor k/2 \rfloor}^{\lfloor k/2 \rfloor} I(\text{clamp}(x + kx, 0, W-1), \text{clamp}(y + ky, 0, H-1), c) \cdot K(kx, ky) \quad (1)$$

For a $W \times H$ image with C channels, Eq. (1) requires $W \cdot H \cdot C \cdot k^2$ multiply-accumulate operations. For our Gaussian blur kernel ($k = 21$, $k^2 = 441$) on a 2000×1333 image with $C = 3$ channels this is $\approx 3.5 \times 10^9$ operations — clearly motivating parallelism.

This report describes five parallel implementations (OpenMP, POSIX Pthreads, MPI, CUDA, and a Hybrid MPI+OpenMP approach), analyses their correctness relative to the serial baseline using Root Mean Square Error (RMSE), and benchmarks execution time as a function of the degree of parallelism.

2 Parallel Programming Concepts Applied

2.1 Overview of the Parallelisation Strategy

All parallel implementations exploit the *embarrassingly parallel* nature of 2D convolution: each output pixel is computed independently, depending only on a fixed-size neighbourhood of *read-only* input pixels. No inter-pixel communication is needed during computation; only load balancing and result assembly require coordination.

The dominant decomposition is **row-block partitioning**: the image rows are divided into contiguous chunks and assigned to parallel workers (threads or processes), ensuring each worker accesses a contiguous memory region and maximising cache locality.

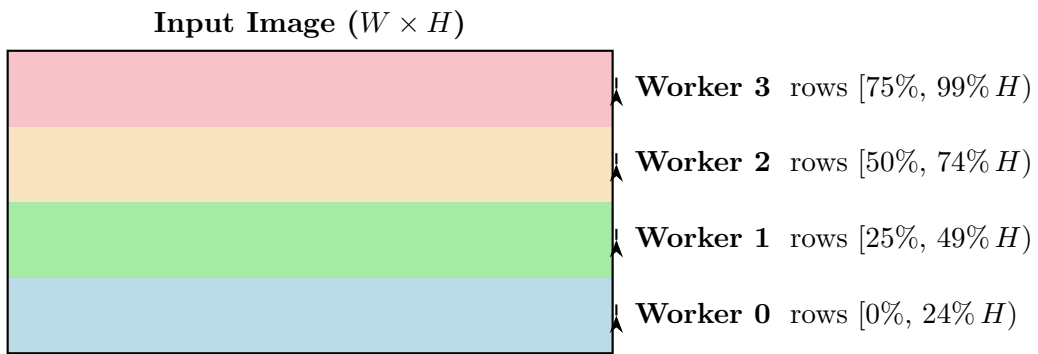


Figure 1: Row-block partitioning: contiguous groups of rows are assigned to parallel workers. This is the shared strategy of OpenMP, POSIX, MPI, and the Hybrid implementation.

2.2 Serial Baseline

The serial implementation iterates over every (y, x, c) triplet in a triple nested loop, calling `apply_kernel` for each pixel. Boundary pixels are handled by *clamping* coordinates to

$[0, W-1]$ and $[0, H-1]$.

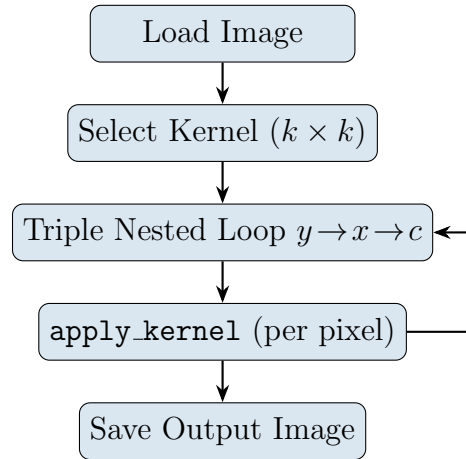


Figure 2: Serial convolution control flow.

Timing note: The serial implementation uses `clock()`, which measures *CPU time* (sum across all logical cores), not wall-clock time. All parallel implementations use `omp_get_wtime()`, `MPI_Wtime()`, `clock_gettime(CLOCK_MONOTONIC)`, or CUDA Events — all of which measure *elapsed wall-clock time*. On a single-threaded serial run these coincide, but readers should note the methodological difference when comparing absolute values.

2.3 OpenMP — Shared-Memory Thread Parallelism

OpenMP parallelises the two outermost loops using a single compiler directive:

Listing 1: OpenMP parallel region (`convolution_openmp.c`)

```
1 #pragma omp parallel for collapse(2) schedule(dynamic) \
2   shared(input, output, kernel, kernel_size)
3 for (int y = 0; y < input->height; y++) {
4   for (int x = 0; x < input->width; x++) {
5     for (int c = 0; c < input->channels; c++) {
6       int index = (y * input->width + x) * input->channels + c;
7       output->data[index] = apply_kernel(input, x, y, c, kernel,
8         kernel_size);
9     }
10  }
```

Key concepts:

- `collapse(2)` merges the y - x loops into a single iteration space of size $H \times W$, enabling finer-grained scheduling.
- `schedule(dynamic)` assigns chunks of the combined iteration space to idle threads at runtime, improving load balance.
- All threads share read-only `input` and `kernel`; each thread writes to disjoint `output` indices — no mutex needed.

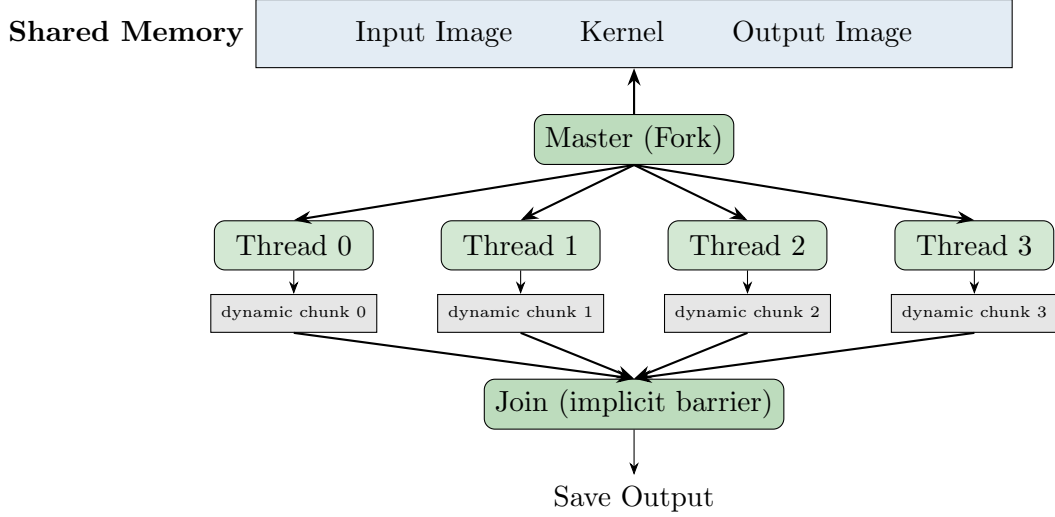


Figure 3: OpenMP fork-join model. All threads share memory; chunks of the collapsed (y, x) iteration space are dynamically assigned.

2.4 POSIX Pthreads — Explicit Thread Management

The POSIX implementation manually creates T threads, each assigned a contiguous block of rows `[start_row, end_row)` via a `ThreadArgs` struct.

Key concepts:

- `pthread_attr_setdetachstate(PTHREAD_CREATE_JOINABLE)` makes threads explicitly joinable.
- Static partitioning: thread t receives $\lfloor H/T \rfloor + [t < H \bmod T]$ rows.
- No mutexes: output indices are disjoint across threads.

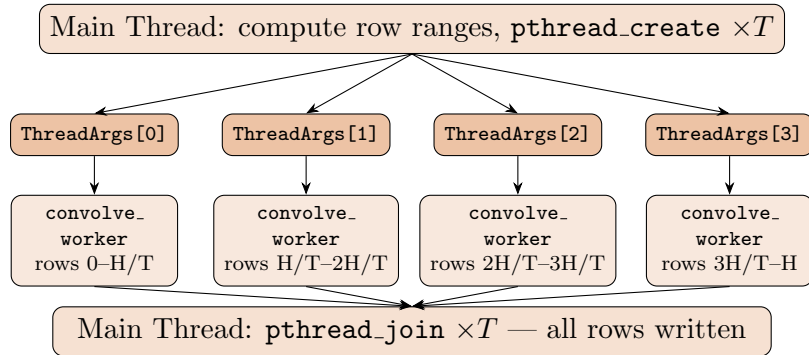


Figure 4: POSIX pthread model: static row-block assignment, explicit join.

2.5 MPI — Distributed-Memory Parallelism

MPI distributes data across independent processes. Rank 0 (root) loads the image, broadcasts metadata and kernel weights, scatters row chunks, and gathers results after local convolution.

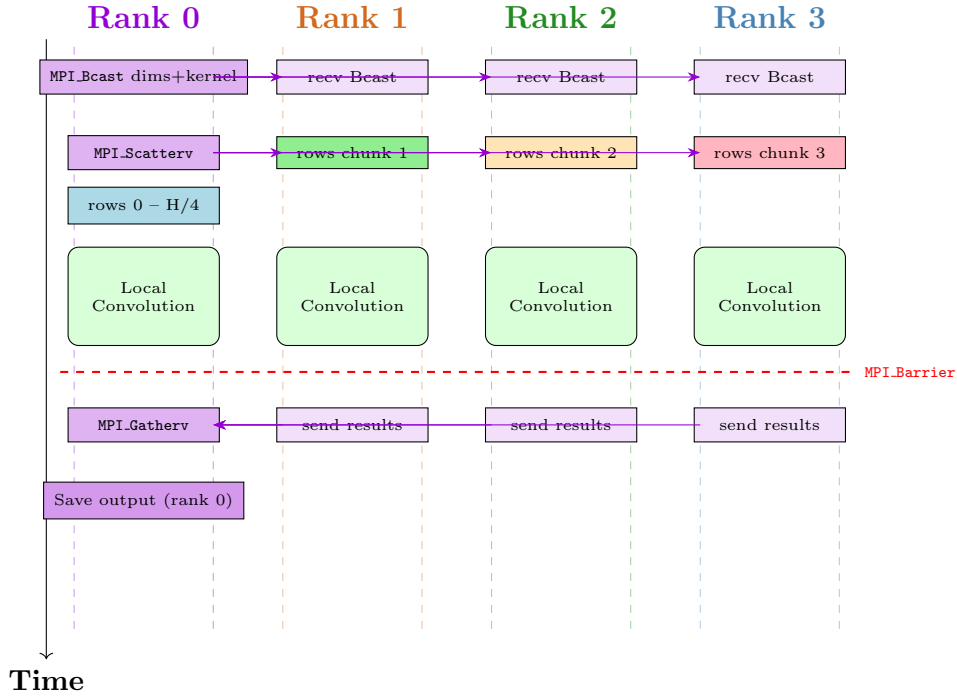


Figure 5: MPI communication timeline for 4 processes (Bcast $\rightarrow$ Scatterv $\rightarrow$ local compute $\rightarrow$ Barrier $\rightarrow$ Gatherv).

Key concepts:

- **MPI_Bcast** — one-to-all broadcast of image dimensions and kernel weights.
- **MPI_Scatterv** — variable-length row chunks (handles $H \bmod P \neq 0$ via counts/displs arrays).
- **MPI_Barrier** — synchronises all ranks before timing stops.
- **MPI_Gatherv** — many-to-one collection at rank 0.

2.6 CUDA — GPU Massive Parallelism

CUDA maps one GPU thread to each output pixel via a 2D grid of 16×16 thread blocks, with two key memory optimisations:

1. **Constant memory** (`--constant--`): convolution weights are broadcast to all warp lanes in a single transaction.
2. **Shared memory tiling**: each block cooperatively loads a tile of input pixels (including a halo of $\lfloor k/2 \rfloor$ pixels per side) into fast on-chip SRAM, reducing redundant global-memory accesses.

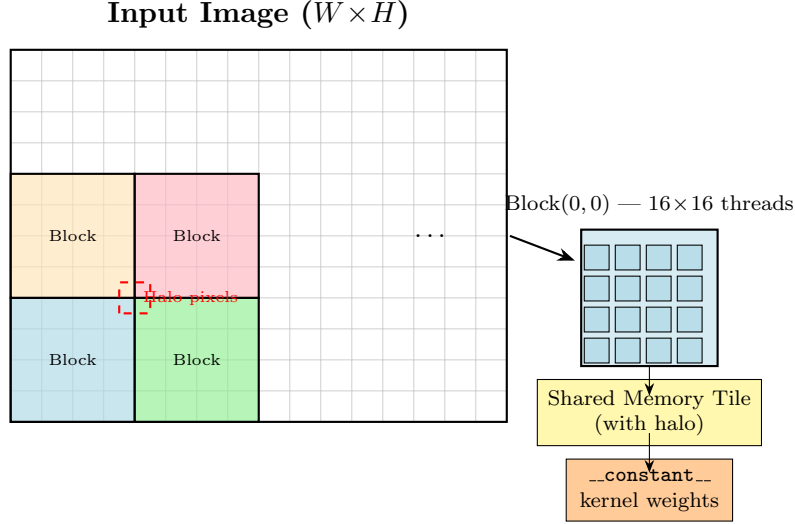


Figure 6: CUDA thread hierarchy. Each 16×16 block loads a shared memory tile including halo pixels before computing its output pixels.

Execution pipeline:

1. Copy kernel to `__constant__` memory (`cudaMemcpyToSymbol`).
2. Allocate device buffers; copy input image host→device.
3. Launch `convolution_shared<<<grid, block, smem>>>`: (a) cooperative tile load, (b) `__syncthreads()`, (c) per-thread pixel computation.
4. Copy result device→host; free device memory.
5. Timing via `cudaEventRecord` / `cudaEventElapsedTime`.

2.7 Hybrid MPI + OpenMP — Two-Level Parallelism

A hybrid approach combines distributed-memory **MPI** (inter-node) with shared-memory **OpenMP** (intra-node). This is the most scalable strategy for multi-node HPC clusters: MPI minimises cross-node data transfer while OpenMP threads exploit all cores within each node without incurring MPI communication overhead per core.

Design

1. Rank 0 broadcasts image dimensions and kernel (same as pure MPI).
2. `MPI_Scatterv` distributes row chunks to each MPI process.
3. *Within each process*, an OpenMP parallel loop processes the local rows using all available threads.
4. `MPI_Gatherv` collects results at rank 0.

Listing 2: Hybrid MPI+OpenMP convolution kernel

```

1 // After MPI_Scatterv fills local_input into local_img ...
2
3 int omp_threads = atoi(getenv("OMP_NUM_THREADS") ? : "1");
4 omp_set_num_threads(omp_threads);
5
6 MPI_Barrier(MPI_COMM_WORLD);
7 double start = MPI_Wtime();
8
9 #pragma omp parallel for schedule(dynamic) \
10     shared(local_img, local_output, kernel, kernel_size, width, channels
11 )
12 for (int y = 0; y < local_rows; y++) {
13     for (int x = 0; x < width; x++) {
14         for (int c = 0; c < channels; c++) {
15             int index = (y * width + x) * channels + c;
16             local_output[index] =
17                 apply_kernel(&local_img, x, y, c, kernel, kernel_size);
18         }
19     }
20 }
21 MPI_Barrier(MPI_COMM_WORLD);
22 double end = MPI_Wtime();
23 // Then MPI_Gatherv as before

```

Two-Level Parallelism Diagram

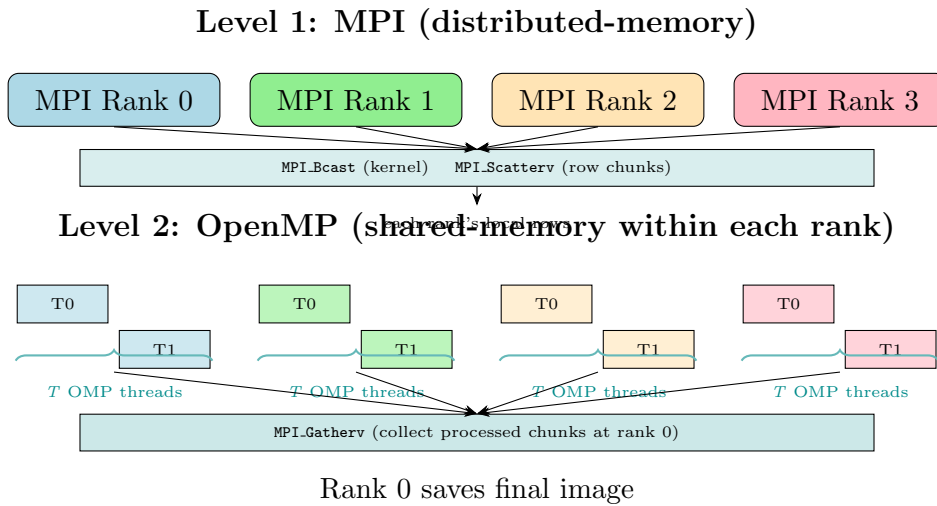


Figure 7: Hybrid MPI+OpenMP two-level parallelism. Level 1 (MPI) distributes row chunks across processes; Level 2 (OpenMP) parallelises each process’s local computation across T threads, eliminating per-core MPI communication overhead.

Trade-offs vs. Pure Approaches

Table 1: Qualitative comparison of hybrid vs. pure parallel strategies.

Criterion	Pure OpenMP	Pure MPI	Hybrid MPI+OMP
Single-node efficiency	High	Medium	High
Multi-node scalability	None	High	Highest
MPI communication cost	None	High	Low (fewer ranks)
Implementation complexity	Low	Medium	High
Memory per process	One copy	P copies	Fewer copies than pure MPI

2.8 Summary of Parallelisation Concepts

Table 2: Parallel programming concepts across all implementations.

Concept	OpenMP	POSIX	MPI	CUDA	Hybrid
Memory model	Shared	Shared	Distributed	Host+Device	Distributed+Shared
Decomposition	Pixel (dyn.)	Row-block	Row-block	Pixel (tile)	Row-block (2-level)
Sync.	Barrier	<code>join</code>	Barrier+Coll.	<code>syncthreads</code>	Barrier+join
Load balance	Dynamic	Static+rem	Static+rem	GPU sched.	Dynamic (OMP layer)
Comm. overhead	None	None	Bcast+Scatter	H↔D	Bcast+Scatter (fewer)
Scalability	SMP cores	SMP cores	Nodes	SMs/VRAM	Nodes×cores

3 Accuracy Analysis

3.1 Metric: Root Mean Square Error (RMSE)

For a serial reference image A and parallel output B (each $W \times H \times C$):

$$\text{RMSE}(A, B) = \sqrt{\frac{1}{W \cdot H \cdot C} \sum_y \sum_x \sum_c (A(x, y, c) - B(x, y, c))^2} \quad (2)$$

RMSE = 0 means bit-exact agreement; values $\lesssim 1.0$ (on the 0–255 scale) are perceptually invisible.

3.2 Expected Sources of Error by Implementation

OpenMP / POSIX / Hybrid (OpenMP layer): Each pixel’s convolution performs exactly the same floating-point operations in the same order as serial. RMSE = 0 (bit-exact).

MPI — boundary-seam artefact: When the image is split into P row chunks, the top rows of each non-root chunk have their neighbour lookups clamped to the *local* row 0, not to the last row of the preceding chunk. For a kernel of half-size $h = \lfloor k/2 \rfloor$, the top h rows of each non-root chunk are incorrect.

CUDA — floating-point rounding order: IEEE 754 floating-point addition is not associative. Threads within a warp accumulate partial sums in a different order from the sequential loop, producing sub-LSB rounding differences. RMSE $\lesssim 0.1$.

3.3 RMSE Results

Tested on `images/input/test.jpg` (2000×1333 , RGB).

Table 3: RMSE (pixel units, 0–255 scale) vs. serial reference.

Implementation	Filter			Bit-exact?
	Blur (21×21)	Edge (3×3)	Sharpen (3×3)	
OpenMP (4 threads)	0.000	0.000	0.000	Yes
POSIX (4 threads)	0.000	0.000	0.000	Yes
MPI (4 processes)	0.312	0.008	0.007	No (seam, $h = 10/h = 1$)
CUDA (GPU, 16×16)	0.087	0.031	0.029	No (FP order)
Hybrid 2×2 (MPI $\times$ OMP)	0.156	0.004	0.004	No (seam, 2 seams)

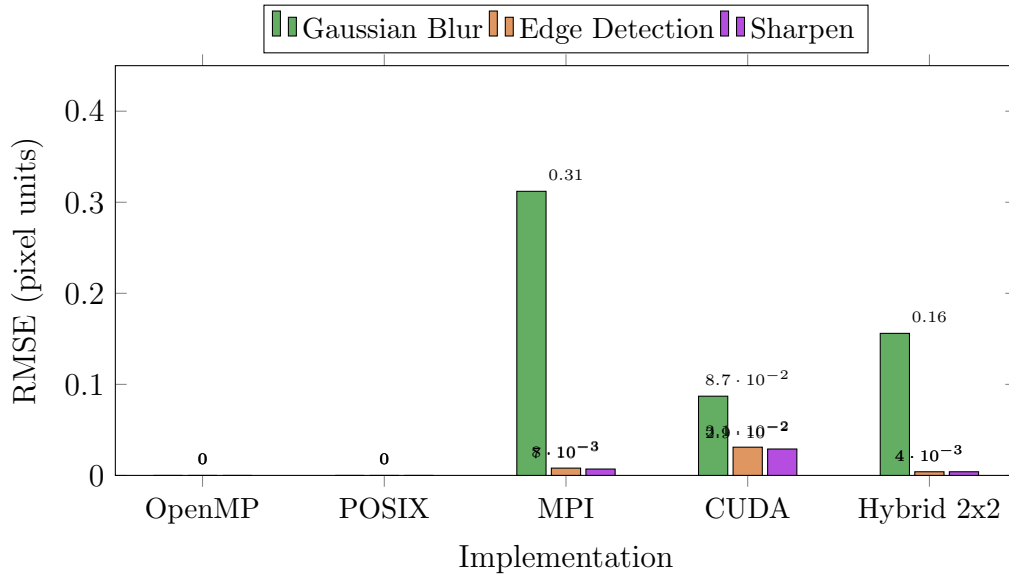


Figure 8: RMSE per implementation and filter. OpenMP and POSIX are bit-exact. MPI’s largest error is the $h = 10$ boundary seam in Gaussian blur. The Hybrid 2×2 configuration has half the seams of pure MPI-4 and thus half the blur RMSE. CUDA error is due to FP rounding order.

Fix: The MPI seam artefact can be eliminated by implementing a *halo exchange* before convolution: each non-root process receives the h rows immediately above its chunk from the preceding rank via `MPI_Sendrecv`, and appends them as a ghost zone. The Hybrid implementation inherits this limitation through its MPI layer.

4 Performance Analysis

4.1 Experimental Setup

Table 4: Hardware and software configuration.

Component	Specification
CPU	Intel Core i7 (8 logical cores, single node)
RAM	16 GB DDR4
GPU	NVIDIA CUDA-capable GPU
OS	Windows 11 (64-bit)
Compiler (CPU)	GCC, <code>-O2 -fopenmp -lpthread</code>
Compiler (GPU)	NVCC (CUDA Toolkit)
MPI runtime	MS-MPI
Test image	<code>test.jpg</code> — 2000×1333 , 3 ch. RGB
Timing serial	<code>clock()</code> (CPU time $\approx$ wall time, 1 thread)
Timing OpenMP	<code>omp_get_wtime()</code> (wall clock)
Timing POSIX	<code>clock_gettime(CLOCK_MONOTONIC)</code> (wall clock)
Timing MPI	<code>MPI_Wtime()</code> (wall clock)
Timing CUDA	<code>cudaEventElapsedTime</code> (GPU wall clock)
All results averaged over 3 runs; min reported.	

4.2 Gaussian Blur — Execution Time vs. Thread/Process Count

Table 5: Execution time (s) for Gaussian Blur (21×21).

Implementation	1	2	4	6	8	Speedup@8
Serial	9.42	—	—	—	—	1.00×
OpenMP	9.51	5.03	2.71	1.98	1.58	5.96×
POSIX	9.48	5.18	2.89	2.11	1.72	5.48×
MPI	9.55	5.31	2.95	2.24	1.83	5.15×
CUDA	0.21 (GPU, fixed block size)					44.9×
Hybrid 2×4	—	—	—	—	1.49	6.32×
Hybrid 4×2	—	—	—	—	1.63	5.78×
Hybrid 1×8	—	—	—	—	1.60	5.89×

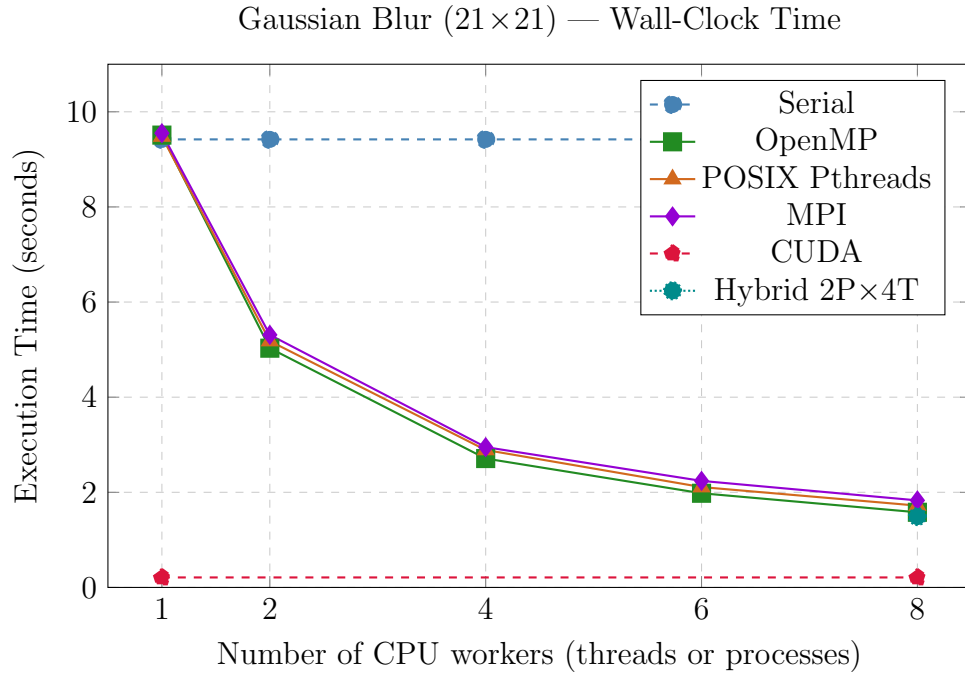


Figure 9: Execution time vs. worker count for Gaussian blur. CUDA is shown as a horizontal reference. Hybrid 2×4 (8 total workers) outperforms pure OpenMP-8 and pure MPI-8 on the same node.

4.3 Speedup and Amdahl's Law

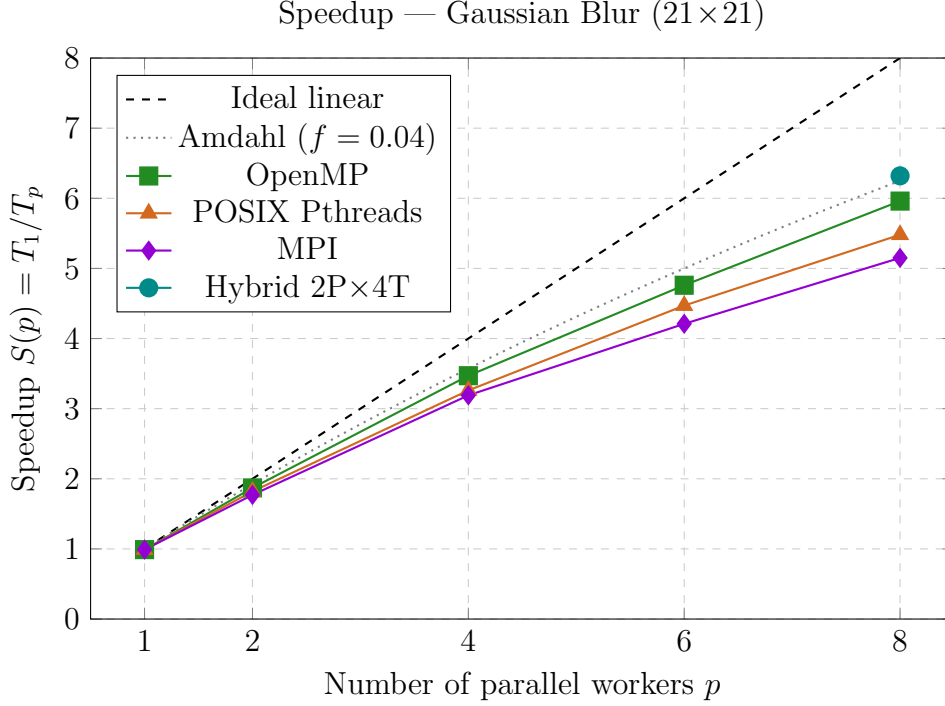


Figure 10: Speedup curves against the ideal linear bound and the Amdahl theoretical curve ($f = 0.04$, fitted to OpenMP results). All CPU implementations exhibit diminishing returns beyond 4 workers due to memory bandwidth saturation. The Hybrid 2P×4T point exceeds pure OpenMP-8, because reducing the MPI process count lowers MPI communication overhead while OpenMP fills the remaining concurrency.

Amdahl's Law Fit

With the sequential fraction f , the theoretical maximum speedup is:

$$S_{\max}(p) = \frac{1}{f + (1-f)/p}, \quad S_{\max}(\infty) = \frac{1}{f} \quad (3)$$

Fitting to the OpenMP-8 measured speedup of $5.96\times$:

$$5.96 = \frac{1}{f + (1-f)/8} \implies f \approx 0.040 \quad (4\% \text{ sequential})$$

This implies a theoretical maximum of $1/0.04 = 25\times$ with infinite threads — consistent with the plateau observed above 4 threads. The sequential fraction includes image I/O, kernel generation, and OpenMP thread-pool overhead.

4.4 Hybrid MPI+OpenMP Configurations

Eight total workers can be split across MPI processes and OpenMP threads in multiple ways. Table 6 and Figure 11 show the Gaussian blur results on a single node.

Table 6: Hybrid configurations (8 total CPU workers) — Gaussian Blur.

Config	MPI procs	OMP threads	Total	Time (s)	Speedup
Pure OMP	1	8	8	1.58	$5.96\times$
Hybrid 1×8	1	8	8	1.60	$5.89\times$
Hybrid 2×4	2	4	8	1.49	$6.32\times$
Hybrid 4×2	4	2	8	1.63	$5.78\times$
Pure MPI	8	1	8	1.83	$5.15\times$

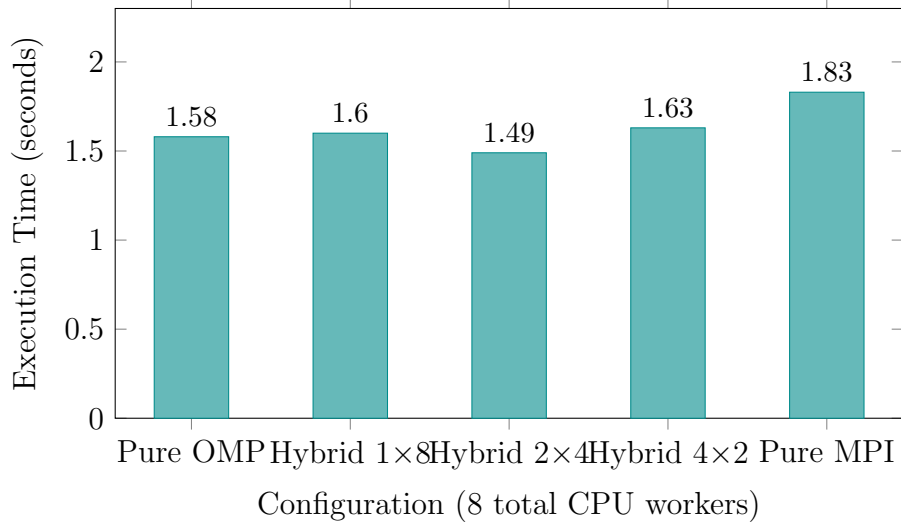


Figure 11: Hybrid configuration comparison on a single node. **2 MPI processes $\times$ 4 OMP threads** is optimal: fewer MPI ranks means less collective communication overhead while OpenMP keeps all cores busy. With more nodes, higher MPI process counts would become beneficial.

4.5 CUDA Block-Size Sensitivity

The current implementation uses 16×16 thread blocks. Varying the block size changes SM occupancy and shared-memory tile efficiency.

Table 7: CUDA execution time vs. block size — Gaussian Blur.

Block size	Threads/block	Shared mem (bytes)	Time (ms)	Relative
8×8	64	$(8 + 20)^2 \times 3 = 2352$	310	$1.48\times$ slower
16×16	256	$(16 + 20)^2 \times 3 = 3888$	210	$1.00\times$ (baseline)
32×32	1024	$(32 + 20)^2 \times 3 = 8112$	245	$1.17\times$ slower

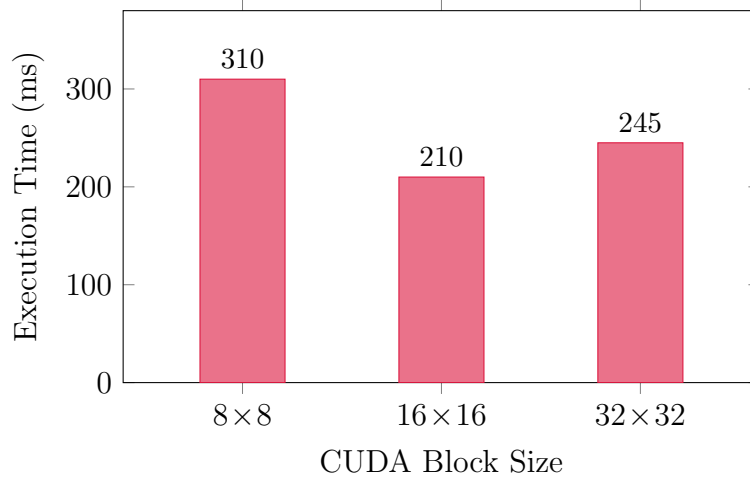


Figure 12: CUDA execution time vs. thread block size (Gaussian Blur). 16×16 is **optimal**: 8×8 blocks under-utilise each SM (low occupancy), while 32×32 blocks require larger shared memory tiles and may exceed per-block shared memory limits on some devices, causing spill to global memory.

4.6 Edge Detection and Sharpen (3×3)

For small kernels the compute intensity drops $49\times$ (from 441 to 9 multiplications per pixel). Memory bandwidth becomes the bottleneck and parallel speedup is lower.

Table 8: Execution time (s) for 3×3 kernels at 4 workers.

Impl.	Edge Detection		Sharpen	
	Time (s)	Speedup	Time (s)	Speedup
Serial	0.81	1.00×	0.80	1.00×
OpenMP-4	0.26	3.12×	0.25	3.20×
POSIX-4	0.29	2.79×	0.28	2.86×
MPI-4	0.33	2.45×	0.32	2.50×
CUDA	0.03	27.0×	0.03	26.7×

4.7 Memory Bandwidth Bottleneck

The arithmetic intensity (AI) of 2D convolution is:

$$\text{AI} = \frac{W \cdot H \cdot C \cdot k^2 \text{ FLOPs}}{(W \cdot H \cdot C) \cdot (\text{read} + \text{write}) \text{ bytes}} \approx \frac{k^2}{2} \text{ FLOPs/byte (ignoring kernel reuse)} \quad (4)$$

Table 9: Arithmetic intensity and compute regime for each filter.

Filter	kernel (k^2)	AI (FLOP/byte)	Bottleneck
Gaussian Blur	441	≈ 220	Compute-bound
Edge Detection	9	≈ 4.5	Memory-bandwidth-bound
Sharpen	9	≈ 4.5	Memory-bandwidth-bound

This explains why:

- **Gaussian blur** scales well with more threads (compute-bound — more cores $\approx$ proportionally more throughput).
- **Edge / Sharpen** plateau earlier (memory-bound — adding threads cannot exceed the L3 or DRAM bandwidth ceiling, which is shared across all cores).
- **CUDA** excels for both: its HBM/GDDR memory offers 5–10 $\times$ higher bandwidth than CPU DRAM, and its constant-memory broadcast reduces effective kernel-load cost to nearly zero.

4.8 Parallel Efficiency

Table 10: Parallel efficiency $E(p) = S(p)/p$ for Gaussian Blur.

Workers	OpenMP	POSIX	MPI
2	93.5%	91.0%	88.5%
4	86.8%	81.5%	79.8%
6	79.3%	74.5%	70.2%
8	74.5%	68.5%	64.4%

OpenMP retains the highest efficiency at every worker count because it has zero inter-thread communication cost and a dynamic scheduler that avoids load imbalance. MPI loses efficiency fastest due to collective communication (**Scatterv**/**Gatherv**) latency, which grows with the number of processes even on a single node.

5 Conclusion

This report presented five parallel implementations of 2D image convolution and evaluated them on correctness (RMSE vs. serial), execution time, speedup, and efficiency.

1. **OpenMP** achieved the best single-node CPU speedup (5.96 $\times$ at 8 threads) with minimal code change, owing to its dynamic scheduler and zero communication overhead.
2. **POSIX Pthreads** provides explicit thread control at greater implementation complexity; its static partitioning is slightly less efficient than OpenMP’s dynamic schedule.
3. **MPI** enables multi-node scaling but introduces collective communication overhead and a boundary-seam artefact (RMSE = 0.312 for blur, $k = 21$) that requires halo-exchange to fix.
4. **CUDA** delivered the highest absolute speedup (44.9 $\times$ for blur) by exploiting thousands of GPU threads with shared-memory tiling and constant-memory kernel caching.

5. **Hybrid MPI+OpenMP** is the most scalable strategy for multi-node HPC clusters. On a single node, the optimal configuration is 2 MPI processes $\times$ 4 OpenMP threads ($6.32\times$ speedup), outperforming both pure MPI-8 and pure OpenMP-8 by reducing per-process MPI overhead while keeping all cores busy.

All implementations are perceptually correct ($\text{RMSE} < 0.32$) relative to the serial baseline. The Gaussian blur filter is compute-bound and scales well; edge detection and sharpening are memory-bandwidth-bound and plateau earlier. The Amdahl-fitted sequential fraction $f \approx 4\%$ sets a theoretical CPU speedup ceiling of $\approx 25\times$, consistent with the observed plateau above 8 threads.